FIFO, LIFO et tampon UART

Comprendre les indices, écrire le C et dimensionner la mémoire

Fiche complémentaire TE1 · 26 octobre 2026

1. FIFO et LIFO : quel élément sort en premier ?

Une structure de stockage définit un **ordre de sortie**. Le tableau est seulement le support mémoire : c'est la manière de gérer les indices qui donne le comportement FIFO ou LIFO.

— **FIFO, First In, First Out** : premier entré, premier sorti. Comme une file à la caisse. Si on ajoute A, puis B, puis C, on lit **A, B, C**.

— **LIFO, Last In, First Out** : dernier entré, premier sorti. Comme une pile d'assiettes. Avec les mêmes ajouts, on lit **C, B, A**. On parle de pile, de *push* et de *pop*.

FIFO · une file

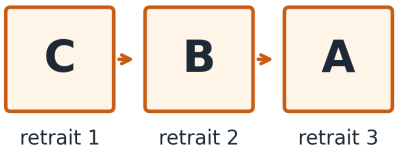
Ajouts successifs : A, puis B, puis C



Le plus ancien sort d'abord.

LIFO · une pile

Ajouts successifs : A, puis B, puis C



Le plus récent sort d'abord.

Pour recevoir une commande UART, l'ordre des caractères compte : une FIFO conserve la chaîne. Une LIFO l'inverserait. Une pile sert plutôt à revenir en arrière, gérer des appels ou parcourir une structure.

Le sens exact de myWrIndex et myRdIndex

Dans la convention du cours, `myWrIndex` désigne la **prochaine case à écrire**, et `myRdIndex` la **prochaine case à lire**. Ce sont des positions, pas les données et pas des compteurs du nombre total d'octets reçus. Leurs valeurs restent entre 0 et $N - 1$.

L'ordre à retenir

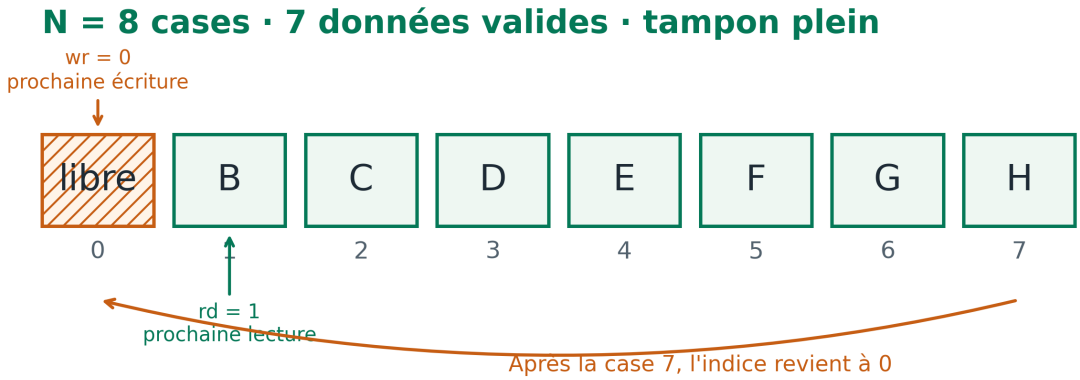
Écrire : vérifier plein, déposer la donnée à `myWrIndex`, puis avancer `myWrIndex`. Lire : vérifier vide, récupérer la donnée à `myRdIndex`, puis avancer `myRdIndex`. On ne déplace pas les autres éléments du tableau.

2. Le tampon circulaire : revenir au début sans déplacer les données

Pour un tableau de N cases, avancer signifie :

$$\text{suivant}(i) = (i + 1) \bmod N$$

Avec $N = 8$, on parcourt $0, 1, 2, 3, 4, 5, 6, 7, 0, 1, \ldots$: $7 + 1 = 8$ et $8 \bmod 8 = 0$. Le tableau reste linéaire en mémoire ; seuls les indices font le tour.



Ordre de lecture : B → C → D → E → F → G → H. La case 0 reste réservée.

Suivre les opérations une par une ($N = 8$)

Opération terminée	wr	rd	Contenu logique FIFO
Initialiser	0	0	vide
Ajouter A, B, C	3	0	A, B, C
Lire une fois : retourne A	3	1	B, C
Ajouter D, E, F, G, H	0	1	B, C, D, E, F, G, H
Tenter d'ajouter I : refus	0	1	inchangé, plein
Lire une fois : retourne B	0	2	C, D, E, F, G, H
Ajouter I en case 0	1	2	C, D, E, F, G, H, I

Pourquoi réserver une case ?

Si les N cases étaient toutes occupées après un tour complet, on retrouverait $wr = rd$, exactement comme à vide. La convention choisie garde donc toujours une case disponible pour lever l'ambiguïté.

$$\text{vide : } wr = rd \qquad \text{plein : } (wr + 1) \bmod N = rd \qquad C_{\text{utile}} = N - 1$$

$$q = (wr + N - rd) \bmod N \qquad \text{places libres} = (N - 1) - q$$

Ici, $wr = 0$ et $rd = 1$ donnent $q = (0+8-1) \bmod 8 = 7$: le tampon est plein. Une autre convention avec un compteur d'occupation ou un indicateur de plein peut utiliser les N cases ; il faut alors adapter les tests et synchroniser cet état supplémentaire.

Une case lue peut encore contenir une ancienne valeur

Lire ne remet pas la case à zéro. Une case est valide selon les indices, pas selon la valeur qu'elle contient. Un octet nul peut être une donnée valide.

3. FIFO en C : une implémentation complète et vérifiable

Le fichier associé `fifo-lifo-revision.c` utilise $N = 16$, donc **15 octets utiles**. Les fonctions retournent `false` en cas d'échec : on ne confond pas une erreur et un octet de valeur zéro. Préconditions : pointeurs valides, initialisation avant usage, accès séquentiels ou protégés comme expliqué en section 7.

```c
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

enum { BUFFER_SIZE = 16 };

typedef struct {
    uint8_t myData[BUFFER_SIZE];
    size_t myWrIndex;  /* prochaine case a ecrire */
    size_t myRdIndex;  /* prochaine case a lire */
} Fifo;

static size_t next_index(size_t index)
{
    return (index + 1u) % BUFFER_SIZE;
}

void fifo_init(Fifo *fifo)
{
    fifo->myWrIndex = 0u;
    fifo->myRdIndex = 0u;
}

bool fifo_is_empty(const Fifo *fifo)
{
    return fifo->myWrIndex == fifo->myRdIndex;
}

bool fifo_is_full(const Fifo *fifo)
{
    return next_index(fifo->myWrIndex) == fifo->myRdIndex;
}

size_t fifo_count(const Fifo *fifo)
{
    return (fifo->myWrIndex + BUFFER_SIZE - fifo->myRdIndex)
           % BUFFER_SIZE;
}

bool fifo_push(Fifo *fifo, uint8_t value)
{
    if (fifo_is_full(fifo)) {
        return false;  /* refus : aucune donnee existante ecrasee */
    }
    fifo->myData[fifo->myWrIndex] = value;
    fifo->myWrIndex = next_index(fifo->myWrIndex);
    return true;
}

bool fifo_pop(Fifo *fifo, uint8_t *value)
{
    if (fifo_is_empty(fifo)) {
        return false;  /* *value reste inchange */
    }
    *value = fifo->myData[fifo->myRdIndex];
    fifo->myRdIndex = next_index(fifo->myRdIndex);
    return true;
}
```

Lire le code avec les indices

`myData[myWrIndex]` reçoit la valeur avant l'avancement de wr. `*value` reçoit la valeur avant l'avancement de rd. Un refus ne change aucun indice ni aucune donnée. L'initialisation des indices suffit ; effacer le tableau est inutile pour déterminer si la FIFO est vide.

4. LIFO en C et variante circulaire des annales

Pile classique : un seul indice suffit

On choisit ici `myTop` égal au **nombre d'éléments**, donc aussi à la prochaine case libre. Vide : $myTop = 0$. Plein : $myTop = N$. Cette pile utilise les N cases. Pour empiler, on écrit puis on incrémente. Pour dépiler, on décrémente puis on lit.

```c
/* LIFO classique : myTop est le nombre d'elements / prochaine case libre. */
typedef struct {
    uint8_t myData[BUFFER_SIZE];
    size_t myTop;
} Lifo;

void lifo_init(Lifo *lifo)
{
    lifo->myTop = 0u;
}

bool lifo_push(Lifo *lifo, uint8_t value)
{
    if (lifo->myTop == BUFFER_SIZE) {
        return false;
    }
    lifo->myData[lifo->myTop] = value;
    ++lifo->myTop;
    return true;
}

bool lifo_pop(Lifo *lifo, uint8_t *value)
{
    if (lifo->myTop == 0u) {
        return false;
    }
    --lifo->myTop;
    *value = lifo->myData[lifo->myTop];
    return true;
}
```

Après les ajouts A, B, C, `myTop` vaut 3 : le premier retrait ramène l'indice à 2, puis lit C. Un cours peut choisir un sommet égal à -1 à vide et $N - 1$ à plein : c'est une autre convention. Il ne faut pas mélanger ses tests avec ceux du code ci-dessus.

Retirer le dernier écrit d'un buffer circulaire

Dans la variante des annales, l'ajout utilise wr comme dans une FIFO, mais le retrait revient à la case **précédant wr**. rd reste la borne de début. La capacité reste $N - 1$ pour cette convention.

```c
/* Variante d'annale : retirer le dernier ecrit d'un buffer circulaire.
 * A utiliser en mode LIFO exclusif, sans producteur concurrent. */
bool circular_lifo_pop(Fifo *buffer, uint8_t *value)
{
    if (fifo_is_empty(buffer)) {
        return false;
    }
    const size_t previous =
        (buffer->myWrIndex + BUFFER_SIZE - 1u) % BUFFER_SIZE;
    *value = buffer->myData[previous];
    buffer->myWrIndex = previous;
    return true;
}
```

Ne pas écrire simplement (wr - 1) % N

Quand wr vaut zéro, il faut obtenir $N - 1$. En C, un calcul signé peut donner $-1 \% N = -1$. Avec les promotions entières, `uint16_t` peut aussi être promu en `int`. La forme `(wr + N - 1) % N` évite ce cas avec les petites tailles utilisées ici.

Cette variante suppose un **mode LIFO exclusif et séquentiel**. Elle modifie wr à la lecture : on ne peut donc pas lui appliquer directement l'organisation FIFO avec un producteur ISR et un consommateur main.

5. UART 9600 bauds, 8N1 et SysTick de 10 ms

Compter les bits réellement transmis

8N1 signifie 8 bits de données, aucune parité et 1 bit de stop. Il faut aussi compter le bit de start. Dans ce format UART binaire, un symbole transporte un bit : 9600 bauds correspondent à 9600 bits/s sur la ligne.

$$b_{\text{trame}} = 1_{\text{start}} + 8_{\text{données}} + 0_{\text{parité}} + 1_{\text{stop}} = 10 \text{ bits/octet}$$

$$R = \frac{9600}{10} = 960 \text{ octets/s} \qquad T_{\text{octet}} = \frac{10}{9600} \text{ s} \simeq 1{,}0417 \text{ ms}$$

Les bits start et stop occupent du temps sur la ligne mais ne sont pas des octets stockés dans la FIFO. Ces valeurs supposent des trames consécutives, sans pause entre les caractères.

Une trame 8N1 = 10 temps de bit



8 bits utiles · 2 bits d'encadrement · durée : 1,0417 ms

Une phase possible : 10 octets terminés en 10 ms



Chaque point = un octet complet reçu · espacement : 1,0417 ms

Combien d'octets peuvent arriver entre deux traitements ?

Un SysTick de 10 ms correspond à 100 périodes par seconde.

$$T = 0{,}010 \text{ s} \qquad Q = RT = 960 \times 0{,}010 = 9{,}6 \text{ octets}$$

On ne stocke pas 0,6 octet : **9,6 est une moyenne**. Selon le décalage entre le début des trames et SysTick, **9 ou 10 octets complets** arrivent dans un intervalle de 10 ms. Le schéma montre une phase où dix fins de trame sont espacées de 1,0417 ms dans l'intervalle. Il faut pouvoir accepter le cas à dix octets.

Hypothèse indispensable

Le calcul suppose que le tampon est entièrement vidé à chaque période, qu'il n'y a pas de reliquat initial et que l'intervalle réel entre vidanges ne dépasse pas 10 ms. La période configurée ne suffit pas si le traitement est retardé ou dure longtemps.

6. Dimensionner la capacité et vérifier le débit de traitement

Distinguer capacité utile et taille du tableau

Pour dix octets à conserver, la convention à une case réservée donne :

$$C_{\text{utile,min}} = \lceil 9{,}6 \rceil = 10 \qquad N_{\text{min}} = 10 + 1 = 11 \text{ cases}$$

Si la taille doit être une puissance de deux, ou si l'on choisit cette taille avec une marge :

$$N = 16 \implies C_{\text{utile}} = 15 \text{ octets} \implies 5 \text{ octets de marge}$$

La réponse attendue dans les choix de l'annale est **16 octets**. Le minimum mathématique, avec cette convention et sans obligation de puissance de deux, est **11 cases physiques**. Le modulo fonctionne aussi avec 11 ; le masque `(i + 1) & (N - 1)` exige une puissance de deux.

Intervalle maximal entre vidanges	Arrivées moyennes	Utile minimal	Tableau minimal
10 ms	9,6	10	11
12 ms (retard inclus)	11,52	12	13
25 ms (40 traitements/s)	24	24	25

Pour 25 ms, une taille de 32 cases donne 31 octets utiles. Le sujet TE1 2025 comporte une contradiction entre « 40 fois par seconde » et « SysTick de 10 ms » ; expliciter l'hypothèse retenue puis calculer, au lieu de confondre 40 Hz et 100 Hz.

La formule générale et ses hypothèses

Avec un flux régulier d'octets terminés espacés de $1/R$, une FIFO initialement vide et une latence maximale réelle T_{max} exprimée en secondes :

$$C_{\text{utile}} \geq \lceil R\, T_{\text{max}} \rceil \qquad N \geq C_{\text{utile}} + 1$$

Ajouter au besoin le reliquat déjà présent, une marge pour la tolérance d'horloge et le retard du traitement, ou une borne de rafale si le matériel transfère les octets par lots. La marge de cinq octets de $N = 16$ n'est pas une garantie contre un retard quelconque. À 960 octets/s, quinze octets représentent environ 15,625 ms de réception ; la phase des arrivées doit toujours être prise en compte.

Une FIFO ne répare pas un consommateur trop lent

Lire un seul octet à chaque SysTick de 10 ms donne seulement :

$$R_{\text{sortie}} = \frac{1}{0{,}010} = 100 \text{ octets/s} \qquad R_{\text{accumulation}} = 960 - 100 = 860 \text{ octets/s}$$

Aucun tampon fini ne tient indéfiniment. Il faut vider plusieurs octets par passage et assurer, en régime continu, un débit de service suffisant ; une marge de débit permet de résorber les retards. Une vidange en boucle est adaptée si le traitement par octet reste rapide et borné.

7. Du schéma UART au programme : ISR, main et concurrence

Le chemin des données est : **ligne UART → réception matérielle → ISR de réception → FIFO logicielle → main**. L'ISR collecte rapidement les octets disponibles, les ajoute à la FIFO et signale les refus. Le programme principal effectue le traitement plus long. SysTick peut seulement donner le rythme, en posant un indicateur ; il n'est pas nécessaire de faire toute la vidange dans son interruption.

```
/* Pseudocode d'integration : API materielle a adapter. */
UART_RX_ISR:
    tant que le materiel a un octet disponible:
        octet = lire_le_materiel()
        si fifo_push(&rx, octet) echoue:
            signaler_un_debordement()

SysTick_ISR:
    signaler_traitement_periodique()

main:
    si traitement_periodique_demande():
        tant que pop_rx_protege(&octet) reussit:
            traiter(octet)
```

Ce bloc est du **pseudocode**, pas une API TivaWare à compiler telle quelle. `pop_rx_protege` signifie que les règles de partage mémoire décrites ci-dessous sont appliquées.

Pourquoi le code élémentaire n'est pas automatiquement sûr en interruption

Le code de la section 3 est séquentiel. Pour le partager entre UART et main, il faut vérifier l'atomicité des indices, l'ordre mémoire et le comportement du compilateur. `volatile` ne garantit pas à lui seul ces propriétés et ne constitue pas une barrière pour les données ordinaires.

Une méthode simple pour cet exemple consiste à protéger chaque accès main à la FIFO par une **section critique très courte** qui empêche l'ISR UART productrice d'intervenir. Sauver puis restaurer l'état antérieur des interruptions ; ne pas les réactiver aveuglément. Retirer l'octet dans la section critique, puis le traiter **après** sa sortie. Utiliser les primitives du support cible, qui doivent aussi imposer l'ordre nécessaire au compilateur.

Une FIFO sans verrou à un producteur et un consommateur est possible : le producteur seul écrit wr, le consommateur seul écrit rd, et les accès sont publiés avec les garanties mémoire appropriées. Les atomiques C11 avec acquire/release constituent une piste, à condition que la chaîne de compilation les prenne en charge pour les interruptions et qu'elles soient réellement sans verrou sur la cible. Ne pas remplacer ce contrat par l'ajout isolé de `volatile`.

L'indicateur posé par SysTick est lui aussi partagé : synchroniser son test et son effacement, par exemple dans une courte section critique. Sinon une interruption entre les deux peut être perdue et allonger le délai réel entre vidanges. Un simple indicateur regroupe plusieurs demandes ; utiliser un compteur synchronisé si chaque occurrence doit être conservée.

Deux débordements différents

Une FIFO logicielle de 16 cases ne protège pas le matériel UART contre une ISR servie trop tard. Vérifier aussi le FIFO ou registre de réception matériel, ses seuils d'interruption, les interruptions masquées et les indicateurs d'erreur UART.

Que faire quand la FIFO est pleine ?

Le code fourni **refuse le nouvel octet** et conserve les anciens. Le refus doit être traité par l'appelant : compteur d'erreurs, arrêt du flux si un contrôle de flux existe, ou resynchronisation du protocole. Écraser le plus ancien est une autre politique, à décider explicitement ; elle modifie aussi rd et complique le partage ISR/main.

8. Mémo des pièges et méthode au TE1

1. Lire la convention des indices avant de raisonner : prochaine case ou dernier élément ?
2. Dessiner les cases numérotées, placer wr et rd, puis exécuter une opération à la fois.
3. Avec une case réservée, écrire les tests vide et plein, puis $C_{\text{utile}} = N - 1$.
4. Pour UART, compter start, données, parité, stop ; convertir les bauds en octets/s.
5. Multiplier par le délai réel entre deux vidanges, arrondir vers le haut, ajouter la case réservée.
6. Si demandé, choisir ensuite la puissance de deux suffisante ; $N = 16$ n'est pas 16 octets utiles.
7. Vérifier que le consommateur peut traiter le flux et que les erreurs plein/vide sont prises en compte.

9. Auto-test

1. On insère A, B, C : que renvoient trois retraits en FIFO puis en LIFO ?
2. Dans une FIFO à case réservée avec $N = 8$, $wr = 0$ et $rd = 1$, est-on plein ou vide ? Combien d'octets ?
3. Après une lecture réussie, faut-il mettre la case à zéro ?
4. Pourquoi 9600 bauds en 8N1 ne donnent-ils pas 1200 octets/s ?
5. Pour 10 ms, pourquoi conserver 10 octets plutôt que 9 ? Quelle taille de tableau minimale ?
6. Que devient un flux de 960 octets/s si main ne retire que 100 octets/s ?
7. Dans un tampon non vide avec $wr = 0$ et $N = 16$, quel est l'indice de la dernière écriture ?
8. L'ajout de `volatile` suffit-il à rendre les fonctions sûres en ISR/main ?

Réponses. 1) FIFO : A, B, C ; LIFO : C, B, A. 2) Plein : $(0 + 1) \bmod 8 = 1 = rd$; sept octets. 3) Non : les indices définissent les cases valides. 4) Il y a dix bits par octet sur la ligne, pas huit : $9600/10 = 960$ octets/s. 5) 9,6 est une moyenne, dix arrivées sont possibles ; onze cases physiques avec une case réservée, ou seize si puissance de deux. 6) La file grossit de 860 octets/s en moyenne et finit par déborder. 7) $(0 + 16 - 1) \bmod 16 = 15$. 8) Non : vérifier la synchronisation, l'atomicité et l'ordre mémoire.

10. Origine, sources et portée de la fiche

Cette fiche reprend les thèmes demandés dans *Explication complète TE1* : FIFO/LIFO, indices circulaires, `myWrIndex`, `myRdIndex` et capacité UART. Lors de l'intégration, les réponses détaillées de cette conversation n'étaient exposées que par des références internes illisibles. Le texte est donc une **reconstitution vérifiée**, à partir de la demande, des supports locaux et des références ci-dessous, pas une transcription intégrale de la conversation.

- Supports du cours ING2 : *Interruptions et FIFO*, diapositives 9 à 14 ; annale TE1 2025, spécification UART ; guides *Guide d'apprentissage TE1* et *Annales des TE1* existants.
- Microchip Developer Help : EUSART/AUSART, format asynchrone et trame.
- Linux Kernel : Circular Buffers, indices, capacité et publication des accès. Cette référence explique le principe ; ses primitives ne sont pas une API pour le TM4C.
- GCC : When is a Volatile Object Accessed ?, limites de volatile.

Code associé : `scripts/fifo-lifo-uart/fifo-lifo-revision.c`, dans le cours E2-EMB1. Les calculs, cas plein/vide, retours circulaires et ordres FIFO/LIFO ont été contrôlés ; le code matériel reste à adapter au projet cible.